

Colab Link: [b22ee088-b22es017.ipynb](https://colab.research.google.com/b22ee088-b22es017.ipynb)

Name	Mahek Vanjani, Rhea Doshi
Roll No	B22EE088,B22ES017
Lab	02

Deep Learning Assignment-2

Objective:

The objective of this assignment is to develop a single Convolutional Neural Network (CNN) capable of classifying CIFAR-100 images into three hierarchical levels:

1. **Individual Class (100 classes):** Fine-grained categories such as “apple,” “dog,” and “truck.”
2. **Superclass (20 classes):** Higher-level groupings like “fruits and vegetables,” “large carnivores,” and “vehicles.”
3. **Synthesized Group (9 groups):** Custom-defined broader categories such as “aquatic animals,” “large animals,” and “man-made articles.”

The model is trained and evaluated across different train-test splits (70:30, 80:20, and 90:10), handling dataset imbalance effectively. Additionally, a hierarchical severity-based misclassification penalty is introduced to improve classification accuracy across different levels.

Dataset:

The dataset used in this assignment is the **CIFAR-100** dataset, which consists of **60,000 32×32 color images**. The dataset is structured into:

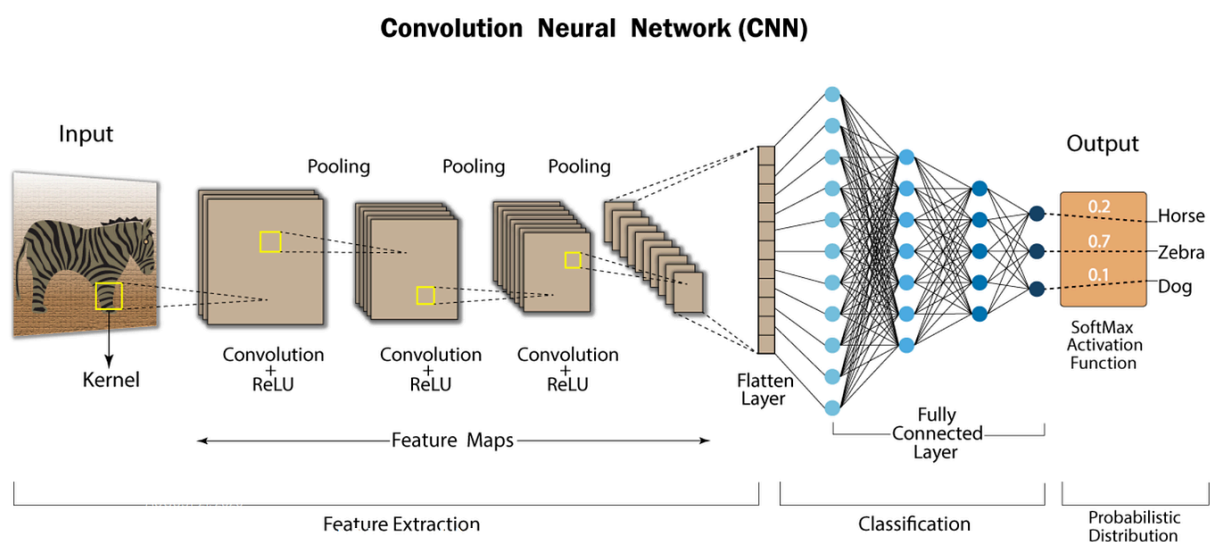
- **100 fine-grained classes**, each containing 600 images.
- **20 superclasses**, grouping similar fine-grained classes.
- **9 synthesized groups**, created based on logical similarities between superclasses.

Due to the imbalance in synthesized groups, special techniques such as class weighting and severity-based penalties are applied.

Visualization:



CNN Model:



Network Architecture:

The CNN model consists of:

1. **Feature Extraction:**

- 3 convolutional layers with increasing filter sizes (64, 128, 256) to learn hierarchical representations.
- Max-pooling layers for downsampling.
- ReLU activation for non-linearity.

2. **Fully Connected Layers:**

- Flattened output from convolutional layers is passed through fully connected layers.
- A shared dense layer is used before branching into separate classification heads.

3. **Output Heads:**

- Three separate classifiers for **individual class (100)**, **superclass (20)**, and **synthesized group (9)** classification tasks.
- Softmax activation in each head for multi-class classification.

Network Layers:

1. **Convolutional Layer 1:**
 - **Input:** $3 \times 32 \times 32 \times 32$ (CIFAR-100 image)
 - **Conv2D:** 32 filters, kernel size = 3×3 , stride = 1, padding = 1
 - **Batch Normalization**
 - **Activation:** ReLU
2. **Convolutional Layer 2:**
 - **Conv2D:** 64 filters, kernel size = 3×3 , stride = 1, padding = 1
 - **Batch Normalization**
 - **Activation:** ReLU
3. **Convolutional Layer 3:**
 - **Conv2D:** 128 filters, kernel size = 3×3 , stride = 1, padding = 1
 - **Batch Normalization**
 - **Activation:** ReLU
4. **Max Pooling Layer:**
 - Kernel size = 2×2 , stride = 2
5. **Fully Connected Layers:**
 - **FC1:** 2048 $\rightarrow$ 512 neurons (ReLU activation)
 - **FC2:** 512 $\rightarrow$ 100 neurons (ReLU activation)
6. **Multi-task classifier heads:**
 - **Fine Category Classifier:** 100 $\rightarrow$ 100 (Softmax activation)
 - **Superclass Classifier:** 100 $\rightarrow$ 20 (Softmax activation)
 - **Group Classifier:** 100 $\rightarrow$ 9 (Softmax activation)

Approach:

The training process started with **data preprocessing** to enhance model generalization and improve feature extraction. The CIFAR-100 dataset was preprocessed using a series of transformations, including:

- **Random Horizontal Flip:** Augments the dataset by randomly flipping images horizontally to introduce variability.
- **Random Crop (with padding = 4):** Crops a random 32×32 section from a padded image, preventing the model from overfitting to fixed spatial features.
- **Normalization:** Pixel values were normalized to a **[-1, 1] range** using a mean and standard deviation of **0.5** for all three channels (RGB). This stabilizes training and speeds up convergence.

For handling categorical labels, **one-hot encoding** was applied to the fine-grained class labels (100 classes).

Training and Hyperparameters:

1. **Dataset Splitting:** The dataset was divided into different train-validation splits: **70:30, 80:20, and 90:10**.
2. **Loss Function:** Categorical Cross-Entropy was used with weighted loss calculation to account for hierarchical imbalance. The weights were:
 - Fine classification: $1/(\log(100))$
 - Superclass classification: $1/(\log(20))$
 - Group classification: $1/(\log(9))$
3. **Optimizer:** Adam optimizer with an initial learning rate of **0.001**.
4. **Learning Rate Scheduler:** ReduceLROnPlateau was used to decrease the learning rate when validation loss plateaued.
5. **Batch Size:** 128
6. **Early stopping:** triggered after 3 epochs without improvement in validation loss.
7. **Training Duration:** 30 epochs

Evaluation:

After training, the model was evaluated on the **test set**, which was never used during training or validation. The best-performing model (from validation) was **loaded from the saved checkpoint**. The model predicted the **fine, superclass, and group labels** for the test images.

These are the accuracies that we achieved:

1. For the 70:30 split

- ✓ Fine Class Accuracy: 44.36%
- ✓ Superclass Accuracy: 46.39%
- ✓ Group Accuracy: 50.70%

2. For the 80:20 split

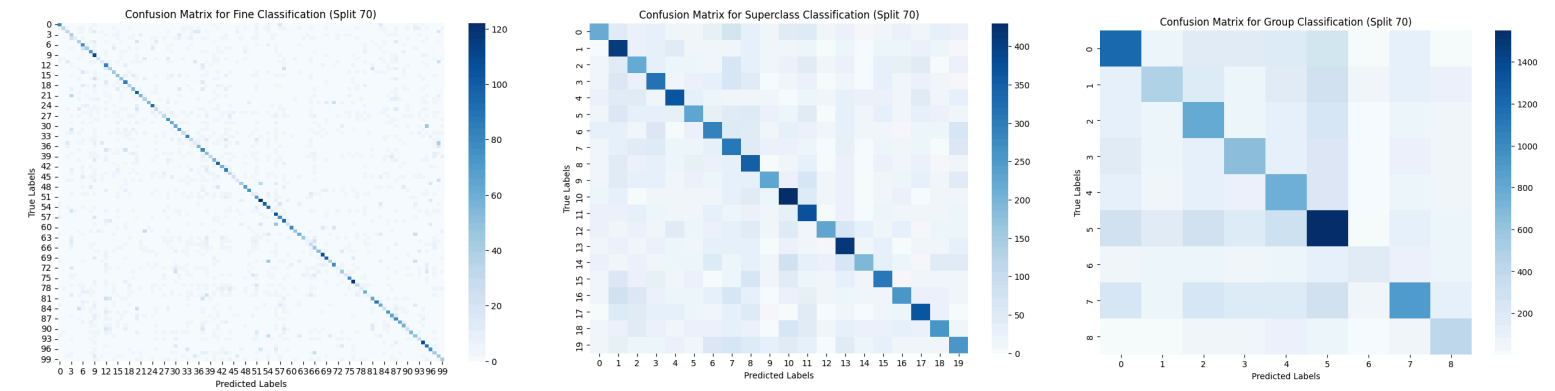
- ✓ Fine Class Accuracy: 45.92%
- ✓ Superclass Accuracy: 47.87%
- ✓ Group Accuracy: 52.82%

3. For the 90:10 split

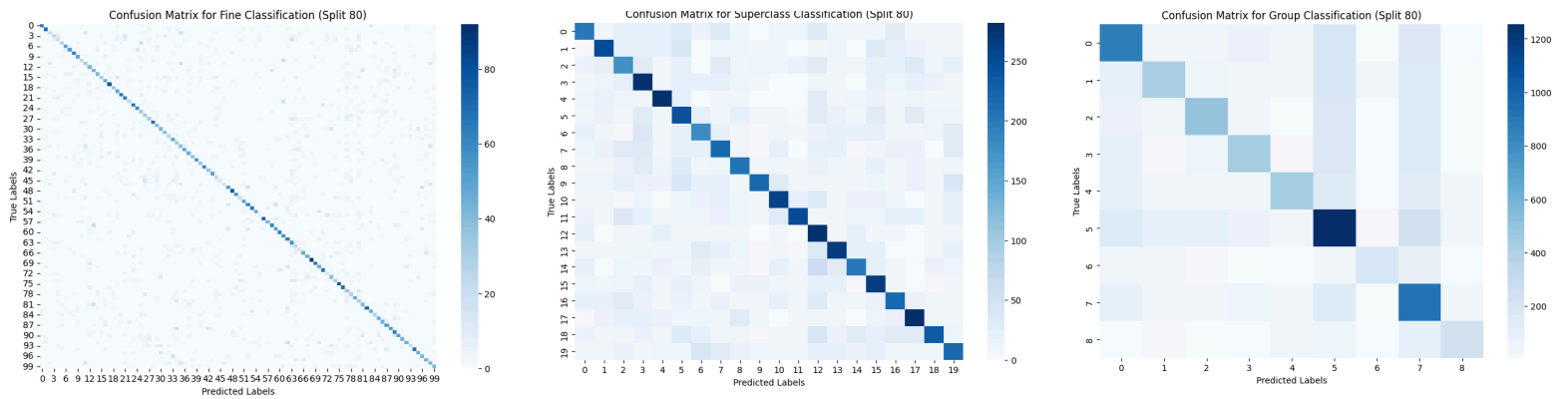
- ✓ Fine Class Accuracy: 41.64%
- ✓ Superclass Accuracy: 43.57%
- ✓ Group Accuracy: 48.69%

Confusion matrix:

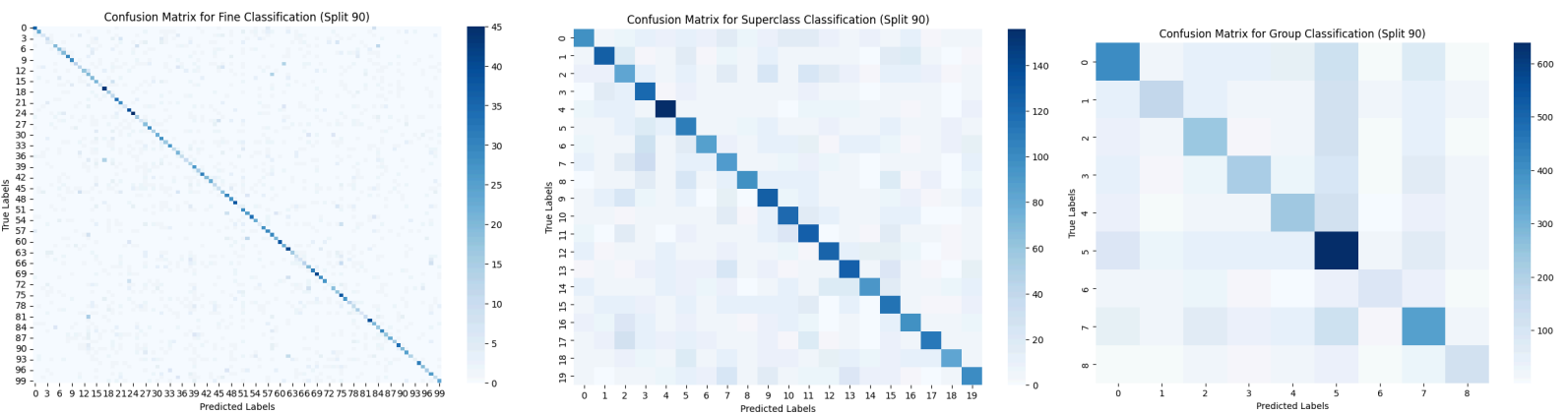
For 70:30 split:



For 80:20 split:



For 90:10 split:



Parameters and Model Summary:

The model consists of a combination of **Convolutional, Batch Normalization, Pooling, and Fully Connected layers**, structured as follows:

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 32, 32]	896
BatchNorm2d-2	[-1, 32, 32, 32]	64
MaxPool2d-3	[-1, 32, 16, 16]	0
Conv2d-4	[-1, 64, 16, 16]	18,496
BatchNorm2d-5	[-1, 64, 16, 16]	128
MaxPool2d-6	[-1, 64, 8, 8]	0
Conv2d-7	[-1, 128, 8, 8]	73,856
BatchNorm2d-8	[-1, 128, 8, 8]	256
MaxPool2d-9	[-1, 128, 4, 4]	0
Linear-10	[-1, 512]	1,049,088
Linear-11	[-1, 100]	51,300
Linear-12	[-1, 100]	10,100
Linear-13	[-1, 20]	2,020
Linear-14	[-1, 9]	909
Total params: 1,207,113		
Trainable params: 1,207,113		
Non-trainable params: 0		
Input size (MB): 0.01		
Forward/backward pass size (MB): 0.99		
Params size (MB): 4.60		
Estimated Total Size (MB): 5.61		

Parameter Summary:

- **Total Parameters: 1,207,113**
- **Trainable Parameters: 1,207,113**
- **Non-trainable Parameters: 0**

Memory Usage:

- **Input Size: 0.01 MB**
- **Forward/Backward Pass Size: 0.99 MB**
- **Parameters Size: 4.60 MB**
- **Estimated Total Size: 5.61 MB**

Bonus Model:

Introduction:

In multi-task classification, standard loss functions do not differentiate between varying degrees of misclassification errors. In this work, we modify the CNN training process to **reduce the severity of misclassification errors** by introducing an additional **severity-based loss function**. The goal is to penalize incorrect predictions based on the severity of the mistake, as defined below:

- **Severity 1:** Misclassification within the same superclass.
- **Severity 2:** Misclassification within the same group but a different superclass.
- **Severity 3:** Misclassification in a different group.

Network Architecture:

The model follows a **feature extractor** consisting of convolutional layers and fully connected layers, followed by a **single classification head** for 100 fine-grained classes.

Network Layers:

- **Feature Extractor:**
 - Conv2D ($3 \rightarrow 64$), kernel = 3×3 , ReLU, MaxPool(2×2)
 - Conv2D ($64 \rightarrow 128$), kernel = 3×3 , ReLU, MaxPool(2×2)
 - Conv2D ($128 \rightarrow 256$), kernel = 3×3 , ReLU, MaxPool(2×2)
- **Fully Connected Layers:**
 - FC1: $256 \times 4 \times 4 \rightarrow 512$
 - FC2: $512 \rightarrow 100$ (Fine Category Classifier)

Severity-Based Loss Function:

We define a custom loss function that assigns penalties based on the severity of misclassification. The severity calculation follows these rules:

1. **Base Penalty:** Any misclassification is penalized.
2. **Superclass Mismatch:** If the predicted class belongs to a different superclass than the true label, an extra penalty is applied.
3. **Group Mismatch:** If the predicted class belongs to a different group than the true label, the highest penalty is applied.

Loss Function Design:

The severity loss function consists of:

- **Cross-Entropy Loss Component:** computes the standard classification loss.
- **Severity Penalty:** Adds penalties based on how severe the misclassification is.

Mathematically, the total loss is defined as:

$$L = -\log P(y_i) + 0.1 \times S$$

where:

- $P(y_i)$ is the probability assigned to the correct class by the model.
- S is the severity penalty:

$$S = 1[\hat{y} \neq y] + 1[\text{super}(\hat{y}) \neq \text{super}(y)] + 2 \times 1[\text{group}(\hat{y}) \neq \text{group}(y)]$$

Training and Hyperparameters:

- **Dataset Splitting:** 90:10 (90% training, 10% validation)
- **Loss Function:** A custom severity-based loss function was implemented to penalize misclassification errors based on severity.
 1. Base Loss: Cross-Entropy
 2. Severity Penalty:
 - Same superclass: +1
 - Same group, different superclass: +2
 - Different group: +3

The final loss formula:

$$L = -\log P(y_i) + 0.1 \times S$$

where S is the severity score.

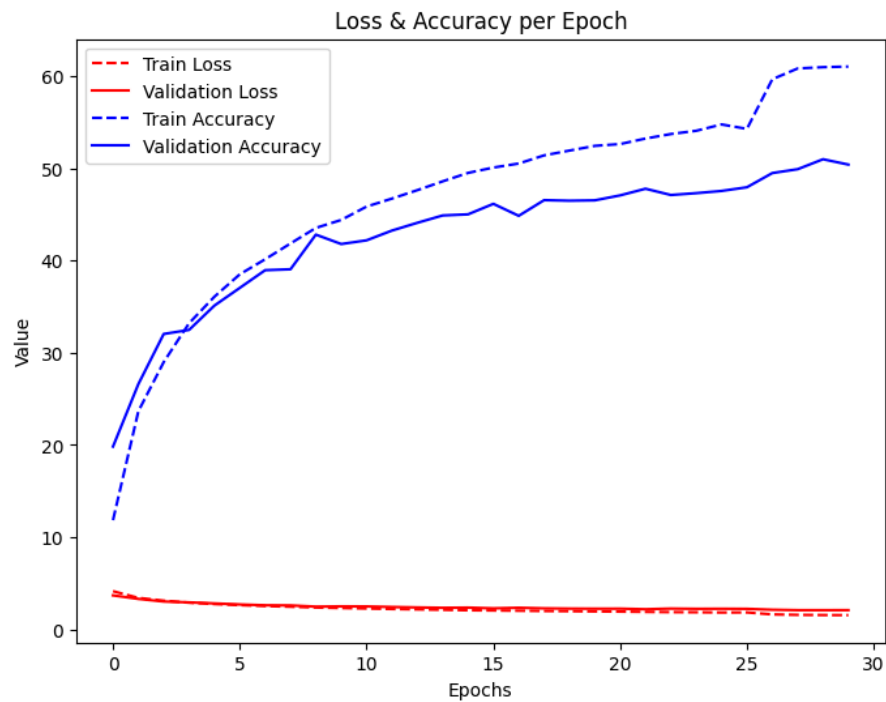
- **Optimizer & Learning Rate Strategy:**
 1. **Optimizer:** Adam optimizer with an initial learning rate of 0.001.
 2. **Learning Rate Scheduler:** ReduceLROnPlateau was used to decrease the learning rate when validation loss plateaued.
- **Batch Size & Regularization:**
 1. **Batch Size:** 128
 2. **Early stopping:** triggered if the validation loss did not improve for 5 consecutive epochs.
- **Training Duration:** The model was trained for a maximum of **30 epochs**, but training was stopped early based on validation loss trends.

Evaluation:

The model was evaluated using a **90:10 train-test split**, ensuring a sufficient amount of data for training while reserving a small portion for validation. The training process spanned **30 epochs**,

and key performance metrics, including **training loss**, **training accuracy**, **validation loss**, and **validation accuracy**, were recorded at each epoch.

- **Best Validation Accuracy: 51.00% at Epoch 29**
- **Final Accuracy: 61.06% (Train), 50.42% (Validation)**
- **Best Validation Loss: 2.0681 at Epoch 29**
- **Final Validation Loss: 2.0708**



Parameters and Model Summary:

The model consists of convolutional layers followed by activation functions, max-pooling layers, and fully connected (linear) layers. Below is the detailed architecture along with the number of trainable parameters:

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 32, 32]	1,792
ReLU-2	[-1, 64, 32, 32]	0
MaxPool2d-3	[-1, 64, 16, 16]	0
Conv2d-4	[-1, 128, 16, 16]	73,856
ReLU-5	[-1, 128, 16, 16]	0
MaxPool2d-6	[-1, 128, 8, 8]	0
Conv2d-7	[-1, 256, 8, 8]	295,168
ReLU-8	[-1, 256, 8, 8]	0
MaxPool2d-9	[-1, 256, 4, 4]	0
Linear-10	[-1, 512]	2,097,664
Linear-11	[-1, 100]	51,300
Total params: 2,519,780		
Trainable params: 2,519,780		
Non-trainable params: 0		
Input size (MB): 0.01		
Forward/backward pass size (MB): 1.97		
Params size (MB): 9.61		
Estimated Total Size (MB): 11.60		

Model Statistics:

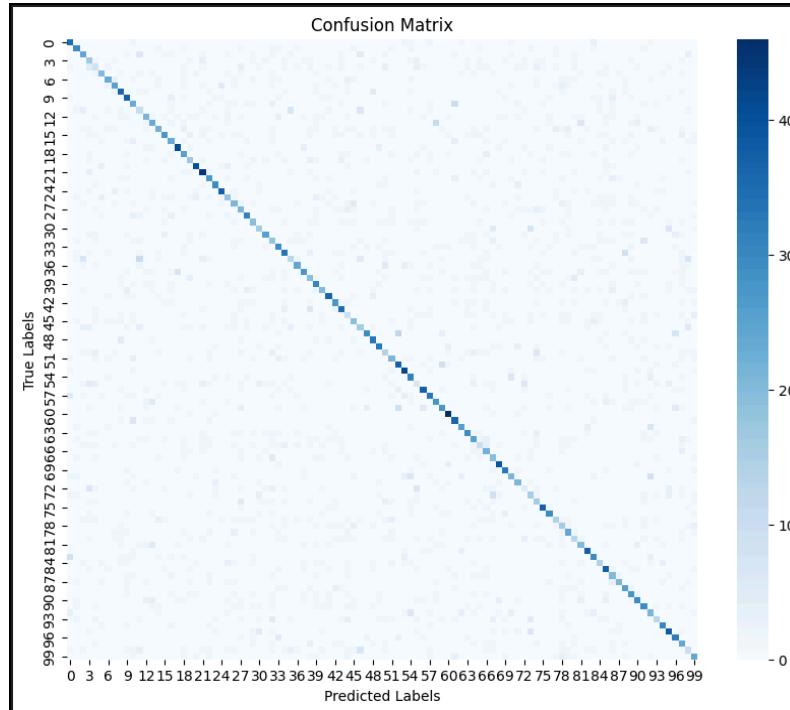
- **Total Parameters:** 2,519,780
- **Trainable Parameters:** 2,519,780
- **Non-Trainable Parameters:** 0

Memory Requirements:

- **Input Size:** 0.01 MB
- **Forward/Backward Pass Size:** 1.97 MB
- **Parameters Size:** 9.61 MB
- **Estimated Total Size:** 11.60 MB

Confusion Matrix:

For 90:10 split, which is the best model for this task, the confusion matrix is as follows:



Conclusion:

In this study, we trained and evaluated two different CNN architectures: **Multi-Head CNN** for the standard classification task and **Single-Head CNN** with a modified loss function to penalize severe misclassifications. The experiments were conducted across multiple train-test splits (**70:30**, **80:20**, and **90:10**), and performance was assessed in terms of accuracy, loss, and confusion matrices.

1. Key insights from the Multi-Head CNN:
 - **Better feature representation** due to multi-task learning.
 - **Improved generalization** by jointly learning multiple related tasks.
 - **Slightly higher computational cost** due to additional classifier heads.
 - **Good performance in classifying superclass and groups**, reducing misclassification severity naturally.
2. Key insights from the Single-Head CNN with severity-based loss:
 - **Reduction in high-severity misclassifications**, making the model more reliable.
 - **Slight trade-off in overall accuracy**, as optimizing for misclassification severity sometimes conflicted with maximizing classification accuracy.
 - **Greater robustness in real-world applications**, where hierarchical relationships are crucial.
 - **Computationally efficient**, as it retained a single classifier head.

Resources:

- https://github.com/Ugenteraan/Deep_Hierarchical_Classification/blob/main/dataset/train.csv
- <https://www.kaggle.com/datasets/fedesoriano/cifar100>
- CNN_Lecture Notes by Andrew Ng.
- <https://github.com/fcakyon/cifar100-resnet>
